


```
config.json: 100% 1.15k/1.15k [00:00<00:00, 87.7kB/s]
model.safetensors: 100% 1.63G/1.63G [00:19<00:00, 143MB/s]
Loading weights: 100% 515/515 [00:00<00:00, 4212.29it/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 2.02kB/s]
vocab.json: 100% 899k/899k [00:00<00:00, 16.5MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 17.3MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 31.6MB/s]
technology 57.7%
course 28.7%
artificial intellignece 10.7%
computer science 2.9%
```

```
import transformers
print(transformers.__file__)
print(transformers.__version__)
```

```
/usr/local/lib/python3.12/dist-packages/transformers/__init__.py
5.10.1
```

```
!pip install --force-reinstall transformers==4.44.2 tokenizers==0.19.1 --break-system-packages
```



```

Collecting transformers==4.44.2
  Downloading transformers-4.44.2-py3-none-any.whl.metadata (43 kB)
    43.7/43.7 kB 1.9 MB/s eta 0:00:00
Collecting tokenizers==0.19.1
  Downloading tokenizers-0.19.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.7 kB)
Collecting filelock (from transformers==4.44.2)
  Downloading filelock-3.29.3-py3-none-any.whl.metadata (2.0 kB)
Collecting huggingface-hub<1.0,>=0.23.2 (from transformers==4.44.2)
  Downloading huggingface_hub-0.36.2-py3-none-any.whl.metadata (15 kB)
Collecting numpy>=1.17 (from transformers==4.44.2)
  Downloading numpy-2.4.6-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting packaging>=20.0 (from transformers==4.44.2)
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting pyyaml>=5.1 (from transformers==4.44.2)
  Downloading pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting regex!=2019.12.17 (from transformers==4.44.2)
  Downloading regex-2026.5.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
    40.4/40.4 kB 2.7 MB/s eta 0:00:00
Collecting requests (from transformers==4.44.2)
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting safetensors>=0.4.1 (from transformers==4.44.2)
  Downloading safetensors-0.8.0-cp310-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.2 kB)
Collecting tqdm>=4.27 (from transformers==4.44.2)
  Downloading tqdm-4.68.2-py3-none-any.whl.metadata (58 kB)
    58.7/58.7 kB 2.1 MB/s eta 0:00:00
Collecting fsspec>=2023.5.0 (from huggingface-hub<1.0,>=0.23.2->transformers==4.44.2)
  Downloading fsspec-2026.4.0-py3-none-any.whl.metadata (10 kB)
Collecting hf-xet<2.0.0,>=1.1.3 (from huggingface-hub<1.0,>=0.23.2->transformers==4.44.2)
  Downloading hf_xet-1.5.1-cp37-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (4.9 kB)
Collecting typing-extensions>=3.7.4.3 (from huggingface-hub<1.0,>=0.23.2->transformers==4.44.2)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting charset_normalizer<4,>=2 (from requests->transformers==4.44.2)
  Downloading charset_normalizer-3.4.7-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (5.6 kB)
    40.9/40.9 kB 2.9 MB/s eta 0:00:00
Collecting idna<4,>=2.5 (from requests->transformers==4.44.2)
  Downloading idna-3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting urllib3<3,>=1.26 (from requests->transformers==4.44.2)
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests->transformers==4.44.2)
  Downloading certifi-2026.5.20-py3-none-any.whl.metadata (2.5 kB)
  Downloading transformers-4.44.2-py3-none-any.whl (9.5 MB)
    9.5/9.5 MB 62.9 MB/s eta 0:00:00
  Downloading tokenizers-0.19.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.6 MB)
    3.6/3.6 MB 80.8 MB/s eta 0:00:00
  Downloading huggingface_hub-0.36.2-py3-none-any.whl (566 kB)
    566.4/566.4 kB 27.8 MB/s eta 0:00:00
  Downloading numpy-2.4.6-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.6 MB)
    16.6/16.6 MB 48.4 MB/s eta 0:00:00
  Downloading packaging-26.2-py3-none-any.whl (100 kB)
    100.2/100.2 kB 7.3 MB/s eta 0:00:00
  Downloading pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (807 kB)
    807.9/807.9 kB 40.5 MB/s eta 0:00:00
  Downloading regex-2026.5.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (801 kB)
    801.2/801.2 kB 42.1 MB/s eta 0:00:00
  Downloading safetensors-0.8.0-cp310-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (516 kB)
    516.0/516.0 kB 30.8 MB/s eta 0:00:00
  Downloading tqdm-4.68.2-py3-none-any.whl (78 kB)
    78.6/78.6 kB 6.4 MB/s eta 0:00:00
  Downloading filelock-3.29.3-py3-none-any.whl (42 kB)
    42.3/42.3 kB 3.0 MB/s eta 0:00:00
  Downloading requests-2.34.2-py3-none-any.whl (73 kB)
    73.1/73.1 kB 5.6 MB/s eta 0:00:00
  Downloading certifi-2026.5.20-py3-none-any.whl (134 kB)
    134.1/134.1 kB 10.5 MB/s eta 0:00:00
  Downloading charset_normalizer-3.4.7-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (216 kB)
    216.6/216.6 kB 15.4 MB/s eta 0:00:00

```

```
from transformers import pipeline
```

```

summariser = pipeline(
    task="summarization",
    model="facebook/bart-large-cnn"
)

```

```
long_text = ""
```

```

The Indian Space Research Organisation (ISRO) successfully launched its
latest Earth observation satellite from the Satish Dhawan Space Centre in
Sriharikota. The satellite carries high-resolution cameras capable of
imaging at 0.5-metre resolution, making it among the most advanced Earth
observation tools deployed. The data will be used for agriculture
monitoring, disaster management, and urban planning.
"""

```

```

summary = summariser(
    long_text,
    max_length=20,
    min_length=15,
    do_sample=False,
)

```

```
from transformers import pipeline
qa = pipeline('question-answering',
model='deepset/bert-base-cased-squad2')
context = '''
Sachin Tendulkar played for India for 24 years from 1989 to 2013.
He scored 100 international centuries and 15,921 Test runs.
He was awarded the Bharat Ratna in 2014.
'''
questions = [
'How many international centuries did Sachin score?',
'When did Sachin Tendulkar retire?',
'What award did he receive in 2014?',
]
```

```

for q in questions:
    answer = qa(question=q, context=context)
    print(f'Q: {q}')
    print(f'A: {answer["answer"]} (confidence: {answer["score"]:.1%})')
# Q: How many international centuries did Sachin score?
# A: 100 (confidence: 94.2%)

```

Some weights of the model checkpoint at deepset/bert-base-cased-squad2 were not used when initializing BertForQuestionAnswering. This is expected if you are initializing BertForQuestionAnswering from the checkpoint of a model trained on another task or to initialize only the parts of a pre-trained model. This is NOT expected if you are initializing BertForQuestionAnswering from the checkpoint of a model that you expect to be initialized with all weights.

Q: How many international centuries did Sachin score?
A: 100 (confidence: 95.2%)
Q: When did Sachin Tendulkar retire?
A: 2013 (confidence: 67.0%)
Q: What award did he receive in 2014?
A: Bharat Ratna (confidence: 97.4%)

Start coding or [generate](#) with AI.

TASK-1

```

from transformers import pipeline

summariser = pipeline(
    task="summarization",
    model="facebook/bart-large-cnn"
)

long_text = """
My name is Harshitha Nethagani. I am currently pursuing a Bachelor of Technology in Artificial Intelligence and Machine Learning. I am a third-year student with a CGPA of 8.3. I completed my Intermediate at Sri Chaitanya Junior College with 75% and my SSC at Sri Sai Vidyanikethan EM School with 10 GPA. I am interested in Machine Learning, Natural Language Processing, Deep Learning, and Data Science. I have worked on projects related to fake job detection and NLP applications. My goal is to build innovative AI solutions and develop my skills in emerging technologies.
"""

summary = summariser(
    long_text,
    max_length=30,
    min_length=15,
    do_sample=False,
    truncation=True
)

print(summary[0]["summary_text"])

```

Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default truncation length is 1024. Harshitha Nethagani is a third-year student at Sri Sai Vidyanikethan EM School. She is

TAAK-2:

```

from transformers import pipeline

qa = pipeline(
    'question-answering',
    model='deepset/bert-base-cased-squad2'
)

context = '''
My name is Harshitha Nethagani.
I am pursuing B.Tech in Artificial Intelligence and Machine Learning.
I am currently in my third year with a CGPA of 8.3.
I completed my Intermediate at Sri Chaitanya Junior College with 75%.
I am interested in Machine Learning, NLP, and Deep Learning.
'''

questions = [
    'What is the name of the student?',
    'Which course is she pursuing?',
    'What is her CGPA?',
    'Where did she complete her Intermediate?',
    'What are her areas of interest?'
]

for q in questions:
    answer = qa(question=q, context=context)

```

```
print(f'Q: {q}')
print(f'A: {answer["answer"]} (confidence: {answer["score"]:.1%})')
```

Some weights of the model checkpoint at deepset/bert-base-cased-squad2 were not used when initializing BertForQuestionAnswering - This IS expected if you are initializing BertForQuestionAnswering from the checkpoint of a model trained on another task or domain - This IS NOT expected if you are initializing BertForQuestionAnswering from the checkpoint of a model that you expect to be initialized for this task

Q: What is the name of the student?
A: Harshitha Nethagani (confidence: 19.7%)

Q: Which course is she pursuing?
A: B.Tech in Artificial Intelligence and Machine Learning (confidence: 5.0%)

Q: What is her CGPA?
A: 8.3 (confidence: 18.1%)

Q: Where did she complete her Intermediate?
A: Sri Chaitanya Junior College (confidence: 46.2%)

Q: What are her areas of interest?
A: Machine Learning, NLP, and Deep Learning (confidence: 69.9%)

```
!pip install transformers sentencepiece sacremoses -q
```

897.5/897.5 kB 13.0 MB/s eta 0:00:00

TASK-3

```
from transformers import pipeline

# English to Telugu Translation
translator = pipeline(
    "translation",
    model="facebook/nllb-200-distilled-600M"
)

texts = [
    "i am N.Harshitha.",
    "I love playing games."
]

for text in texts:
    translated = translator(text,src_lang="eng_Latn",tgt_lang="tel_Telu")[0]["translation_text"]

    print(f"EN: {text}")
    print(f"TE: {translated}")
```

EN: i am N.Harshitha.
TE: నా పేరు ఎన్. హర్షిథా.
EN: I love playing games.
TE: నేను గేమ్స్ ప్లే ప్రేమ.

```
import pandas as pd
from transformers import pipeline

reviews_data = [
    # Bollywood
    {'movie': '3 Idiots', 'source': 'Bollywood',
     'review': 'An absolute masterpiece! Makes you laugh and cry. '
     'Aamir Khan delivers one of his finest performances ever.'},
    {'movie': '3 Idiots', 'source': 'Bollywood',
     'review': 'Overrated and preachy. The message was fine but '
     'execution felt too theatrical and unrealistic.'},
    {'movie': 'Dangal', 'source': 'Bollywood',
     'review': 'Incredibly inspiring! Geeta and Babita brought tears '
     'to my eyes. Best sports film ever made in India.'},
    {'movie': 'Dangal', 'source': 'Bollywood',
     'review': 'Good film but very long. Some scenes dragged unnecessarily.'},
    # Hollywood
    {'movie': 'Inception', 'source': 'Hollywood',
     'review': 'Mind-bending and visually stunning. Nolan at his absolute best.'},
    {'movie': 'Inception', 'source': 'Hollywood',
     'review': 'Too complicated. I had no idea what was happening half the time.'},
    {'movie': 'Parasite', 'source': 'Hollywood',
     'review': 'Perfect film. The class commentary is brilliant. Deserved every Oscar.'},
    {'movie': 'Parasite', 'source': 'Hollywood',
     'review': 'I expected more from an Oscar winner. Ending felt rushed.'},
]
df = pd.DataFrame(reviews_data)
```

```
# Load once - not inside a loop (that would reload weights every iteration)
sentiment = pipeline('sentiment-analysis',
model='distilbert-base-uncased-finetuned-sst-2-english')
# Batch all reviews at once - faster than one-by-one
results = sentiment(df['review'].tolist())
df['sentiment'] = [r['label'] for r in results] # POSITIVE / NEGATIVE
df['confidence'] = [r['score'] for r in results] # 0.0 to 1.0
```

```

# sentiment_score: 1.0 = very positive, 0.0 = very negative
df['sentiment_score'] = df['confidence'].where(
    df['sentiment'] == 'POSITIVE',
    1 - df['confidence']
)
print(df[['movie', 'sentiment', 'confidence']].to_string(index=False))

```

config.json: 100%

629/629 [00:00<00:00, 7.53kB/s]

model.safetensors: 100%

268M/268M [00:06<00:00, 47.0MB/s]

tokenizer_config.json: 100%

48.0/48.0 [00:00<00:00, 3.99kB/s]

vocab.txt: 232k/? [00:00<00:00, 8.31MB/s]

```

/usr/local/lib/python3.12/dist-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean_up_tokenization_
warnings.warn(

```

```

movie sentiment confidence
3 Idiots POSITIVE 0.999879
3 Idiots NEGATIVE 0.999016
Dangal POSITIVE 0.999878
Dangal NEGATIVE 0.992343
Inception POSITIVE 0.999849
Inception NEGATIVE 0.999590
Parasite POSITIVE 0.999857
Parasite NEGATIVE 0.998913

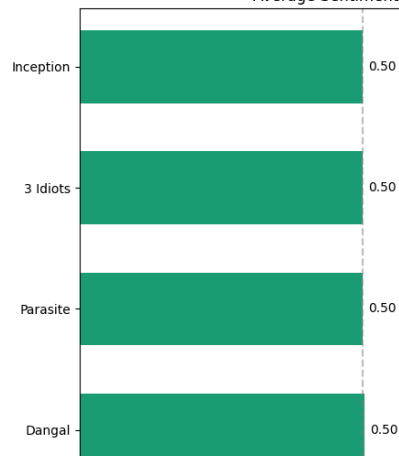
```

```

import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
# Chart 1: Average sentiment score per movie
ax1 = axes[0]
movie_scores = df.groupby('movie')['sentiment_score'].mean().sort_values(ascending=False)
colours = ['#1D9E75' if v >= 0.5 else '#D85A30' for v in movie_scores.values]
bars = ax1.barh(movie_scores.index, movie_scores.values, color=colours,
height=0.6)
ax1.set_xlim(0, 1)
ax1.axvline(0.5, color='gray', linestyle='--', alpha=0.5)
ax1.set_title('Average Sentiment by Movie')
for bar, val in zip(bars, movie_scores.values):
    ax1.text(val+0.01, bar.get_y()+bar.get_height()/2, f'{val:.2f}',
va='center')
# Chart 2: Positive vs Negative split by source
ax2 = axes[1]
for source, colour in [('Bollywood', '#7F77DD'), ('Hollywood', '#1D9E75')]:
    subset = df[df['source'] == source]
    pos = (subset['sentiment'] == 'POSITIVE').sum()
    neg = (subset['sentiment'] == 'NEGATIVE').sum()
    ax2.bar(source, pos, color=colour, alpha=0.9)
    ax2.bar(source, neg, bottom=pos, color=colour, alpha=0.4)
    ax2.set_title('Positive vs Negative by Source')
plt.tight_layout()
plt.savefig('sentiment_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

```

Average Sentiment by Movie



Positive vs Negative by Source

